

Statistical Computing

Visualization of Multivariate Data

Spring 2026

Outline

Introduction

Panel Displays

Correlation Plots

Surface Plots and 3D Scatter Plots

Contour Plots

Other 2D Representations of Data

Principal Components Analysis

Introduction

- Visualization of multivariate data is related to exploratory data analysis (EDA) and statistical graphics.
- “Exploratory” is in contrast to “confirmatory” (e.g., hypothesis testing).
- Exploratory work is useful before hypothesis testing to learn appropriate questions and methods.
- With multivariate data, we may also be interested in dimension reduction or finding structure/groups.
- Here we restrict attention to methods for visualizing multivariate data.

Panel Displays

- A panel display is an array of two-dimensional graphical summaries of pairs of variables.
- A scatterplot matrix displays scatterplots for all pairs of variables in an array.
- Along the diagonal, one may display variable names or other summaries (e.g., densities).
- Before plotting, one may standardize each one-dimensional sample.
- Panel displays can also include arrays of three-dimensional plots.

Python Code: Scatterplot Matrix with Diagonal Densities

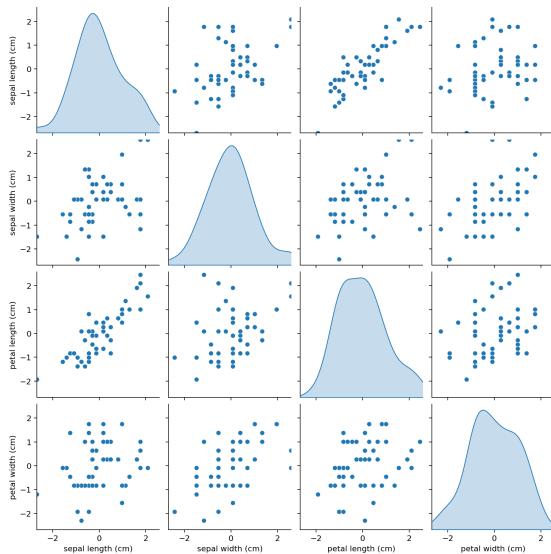
```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
df = iris.frame.iloc[100:150, :4]
x = (df - df.mean()) / df.std(ddof=0)

g = sns.pairplot(x, diag_kind="kde")

plt.show()
```

Figure: Scatterplot Matrix with Diagonal Densities



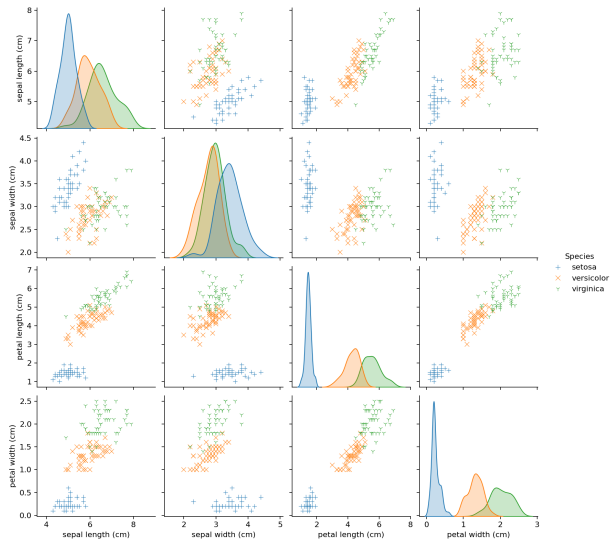
Python Code: Scatterplot Matrix by Species

```
df = iris.frame.rename(columns={"target": "Species"})
df["Species"] = df["Species"].map({0: "setosa", 1: "versicolor", 2: "virginica"})

sns.pairplot(
    df,
    vars=df.columns[:4],
    hue="Species",
    markers=["+", "x", "1"],
)

plt.show()
```

Figure: Scatterplot Matrix by Species



Correlation Plots

Correlation Plots

- With more than about four or five quantitative variables, an array of scatter plots is not helpful for pairwise association.
- A correlation matrix can always be computed, but a large array of correlations is difficult to interpret.
- A graphical summary of pairwise correlations can be very useful.
- (Python note) A direct one-function equivalent of `corrplot` is uncommon; here we use a correlation heatmap.

Python Code: Correlation Plot for Decathlon Events

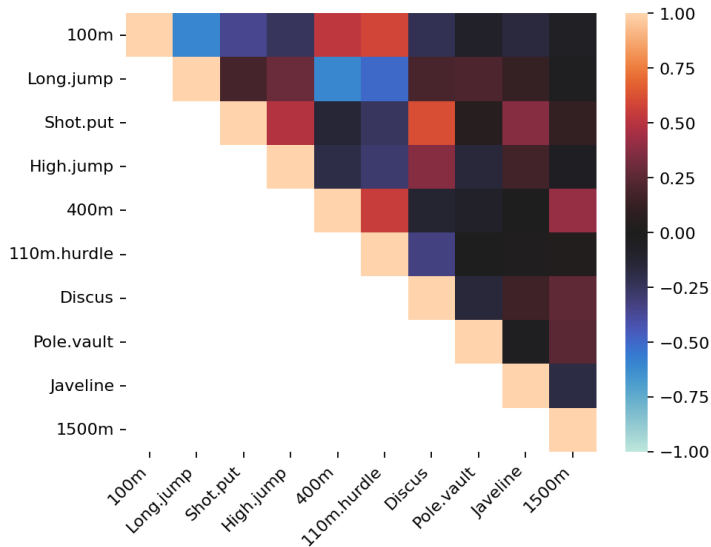
```
import pandas as pd
import numpy as np

decathlon = pd.read_csv("data/decathlon.csv", index_col=0)
decathlon
corr = decathlon.iloc[:, :10].corr()

mask = np.tril(np.ones_like(corr, dtype=bool), k=-1)
ax = sns.heatmap(corr, mask=mask, vmin=-1, vmax=1, center=0)
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha="right")

plt.show()
```

Figure: Correlation Plot of Decathlon Performance



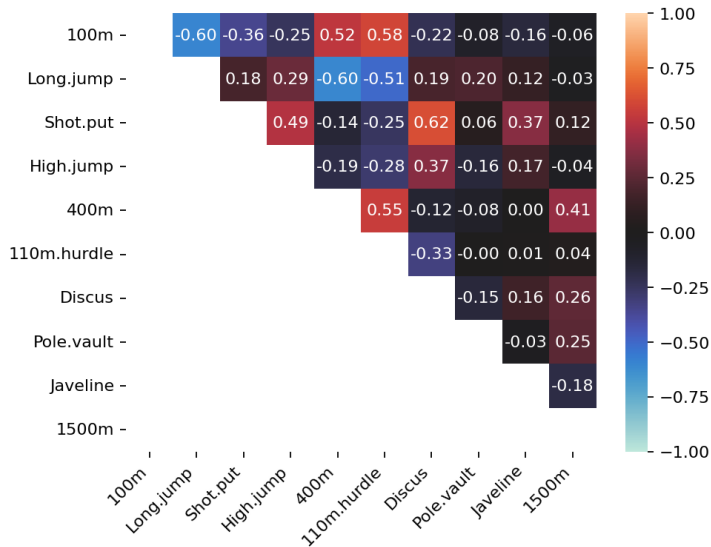
- The correlation plot makes it easy to identify large correlations.
- There are high correlations between pairs of field events like shot put and discus, and between pairs of shorter distance races and hurdles.
- The correlations between pairs of a track event and a field event are small.
- The long jump is negatively correlated with the 400 meter run and 110 meter hurdles.

Python Code: Correlation Plot with Coefficients (No Diagonal)

```
mask = np.tril(np.ones_like(corr, dtype=bool))
ax = sns.heatmap(corr, mask=mask, vmin=-1, vmax=1, center=0,
                 annot=True, fmt=".2f"
                )
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha="right")

plt.show()
```


Figure: Correlation Plot with Correlation Coefficients



Surface Plots and 3D Scatter Plots

Surface Plots and 3D Scatter Plots

- Several tools provide surface and contour plots; a perspective plot draws a surface over the plane.
- For some graphs we mesh a grid of regularly spaced points in the plane.
- To compute function values on a grid, we can evaluate a bivariate function on all (x_i, y_j) pairs.
- We also consider three-dimensional scatterplots to explore possible groups or clusters in data.

- Example: plot the standard bivariate normal density

$$f(x, y) = \frac{1}{2\pi} \exp\left(-\frac{1}{2}(x^2 + y^2)\right), \quad (x, y) \in \mathbb{R}^2.$$

- We create vectors of x and y values, then compute a full grid of $z_{ij} = f(x_i, y_j)$.
- (Python note) Use a meshgrid and vectorized evaluation (outer-style computation via broadcasting).

Python Code: Perspective Plot of Standard BVN Density

```
x = y = np.linspace(-3, 3, 80)
X, Y = np.meshgrid(x, y)
Z = (1/(2*np.pi)) * np.exp(-0.5*(X**2 + Y**2))

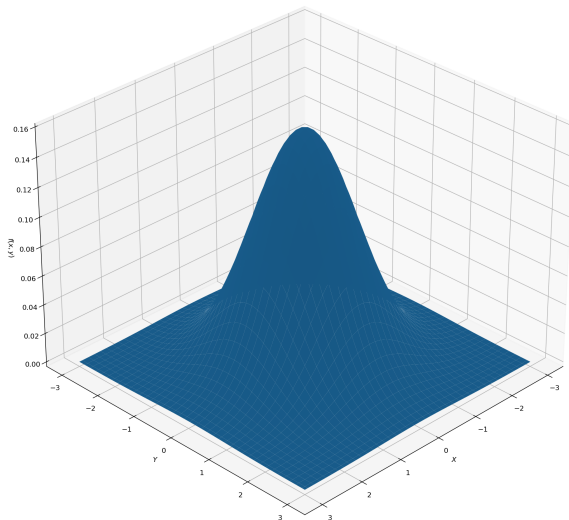
fig = plt.figure(figsize=(15,24))
ax = fig.add_subplot(projection="3d")

ax.plot_surface(X, Y, Z)

ax.view_init(elev=30, azimuth=45)
ax.set_xlabel(r"$X$")
ax.set_ylabel(r"$Y$")
ax.set_zlabel(r"$f(x, y)$")

plt.show()
```

Figure: Perspective Plot of Standard BVN Density



Other Functions for Graphing Surfaces

- Surfaces can also be displayed as a wireframe (mesh) plot.
- Interactive 3D displays allow rotation/tilting to view the surface from different angles.

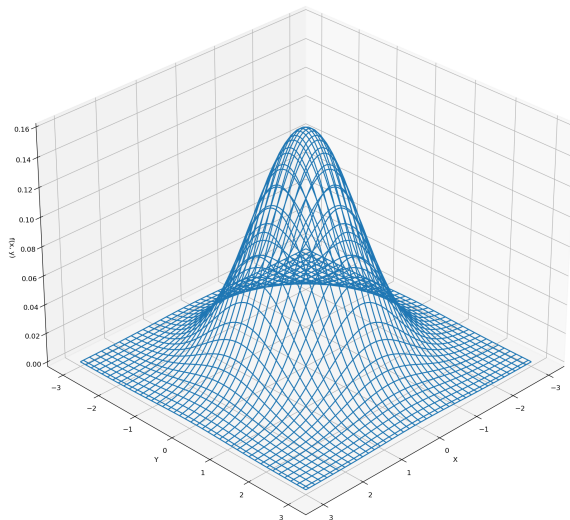
Python Code: Wireframe Surface Plot

```
fig = plt.figure()
fig = plt.figure(figsize=(15,24))
ax = fig.add_subplot(projection="3d")
ax.plot_wireframe(X, Y, Z)

ax.view_init(elev=30, azim=45)
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("f(x, y)")

plt.show()
```


Figure: Wireframe Surface Plot



Three-dimensional Scatterplot

- A 3D scatterplot can help explore whether groups or clusters exist in multivariate data.
- Example: the iris data (three species, four measured variables).
- Plot a 3D scatterplot of three variables with species indicated by plotting symbols.

Python Code: 3D Scatterplot of Iris Data (Grouped)

```
df = iris.frame
species = iris.target

X = df["sepal length (cm)"].to_numpy()
Y = df["sepal width (cm)"].to_numpy()
Z = df["petal length (cm)"].to_numpy()

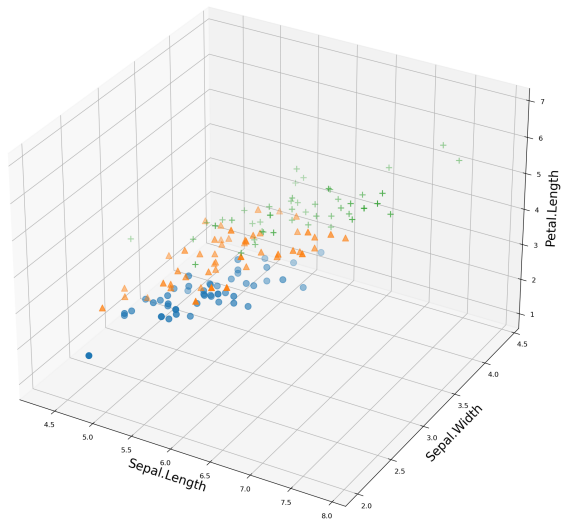
markers = {0: "o", 1: "^", 2: "+"}

fig = plt.figure(figsize=(15,20))
ax = fig.add_subplot(projection="3d")

for k in [0, 1, 2]:
    idx = (species == k)
    ax.scatter(X[idx], Y[idx], Z[idx], marker=markers[k], s=80)

ax.set_xlabel("Sepal.Length", fontsize=18)
ax.set_ylabel("Sepal.Width", fontsize=18)
ax.set_zlabel("Petal.Length", fontsize=18)
```

Figure: 3D Scatterplot of Iris Data



Panel Display of 3D Scatterplots

- With four variables, there are four subsets of three variables to graph.
- Arrange four 3D scatterplots in a 2×2 panel display.
- Use a fixed viewing orientation across panels to compare structure.

```
def scatter_grouped(ax, x, y, z):  
    for k in [0, 1, 2]:  
        idx = (species == k)  
        ax.scatter(x[idx], y[idx], z[idx], marker=markers[k])  
    ax.set_xticks([]); ax.set_yticks([]); ax.set_zticks([])
```

Python Code: Four 3D Scatterplots in a Panel Display

```
fig = plt.figure()
ax1 = fig.add_subplot(221, projection="3d")
scatter_grouped(ax1, df["petal length (cm)"], df["petal width (cm)"], df["sepal length (cm)"])
ax1.view_init(elev=30, azimuth=-75)

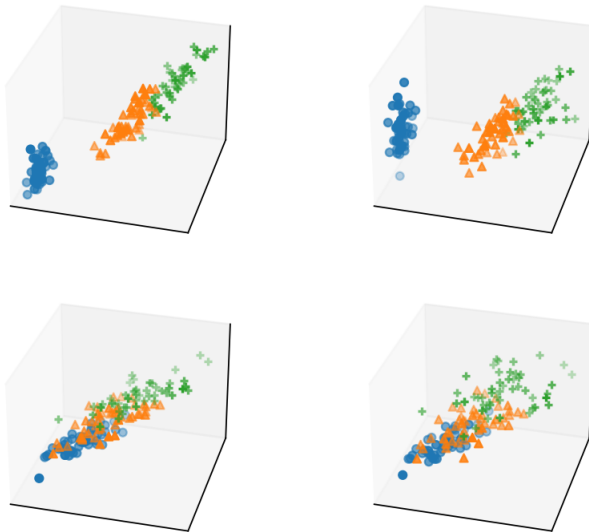
ax2 = fig.add_subplot(222, projection="3d")
scatter_grouped(ax2, df["petal length (cm)"], df["petal width (cm)"], df["sepal width (cm)"])
ax2.view_init(elev=30, azimuth=-75)

ax3 = fig.add_subplot(223, projection="3d")
scatter_grouped(ax3, df["sepal length (cm)"], df["sepal width (cm)"], df["petal length (cm)"])
ax3.view_init(elev=30, azimuth=-75)

ax4 = fig.add_subplot(224, projection="3d")
scatter_grouped(ax4, df["sepal length (cm)"], df["sepal width (cm)"], df["petal width (cm)"])
ax4.view_init(elev=30, azimuth=-75)

plt.tight_layout()
plt.show()
```

Figure: Panel Display of Four 3D Scatterplots



- The three species of iris are separated into groups or clusters in the three-dimensional subspaces spanned by any three of the four variables.
- There is some structure evident in these plots.
- One might follow up with cluster analysis or principal components analysis to analyze the apparent structure in the data.

Contour Plots

- A contour plot represents a 3D surface $(x, y, f(x, y))$ in the plane by projecting level curves $f(x, y) = c$.
- Contour plots and filled contour plots are common variants; contours are often labeled by default.
- A variation uses color to identify contour levels (image-style display).

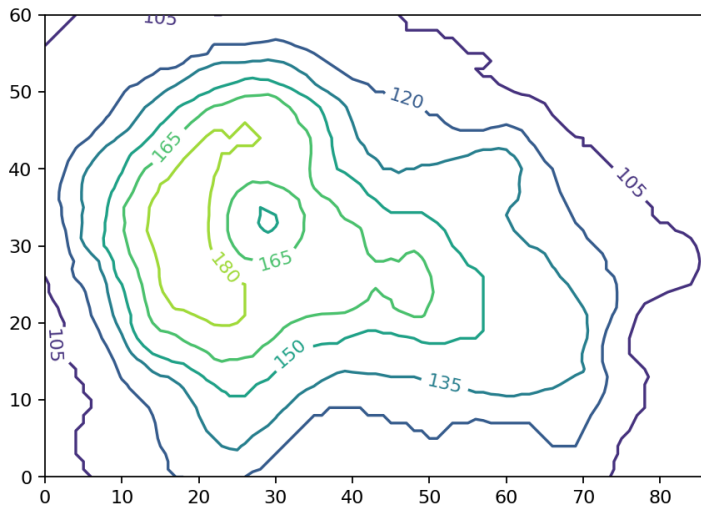
Python Code: Contour Plot with Labels (Volcano Data)

```
volcano = pd.read_csv("data/volcano.csv", index_col=0).T

plt.figure()
cs = plt.contour(volcano)
plt.clabel(cs, inline=True)

plt.gca().set_aspect("equal")
plt.show()
```

Figure: Contour Plot of Volcano Topography



3D View of the Volcano Surface (for Comparison)

- It can be interesting to compare the contour plot with a 3D surface display of the volcano data.
- An interactive 3D view allows rotation/tilting to view the surface from different angles.
- Another 3D view with shading (to indicate contour levels) can also be produced.

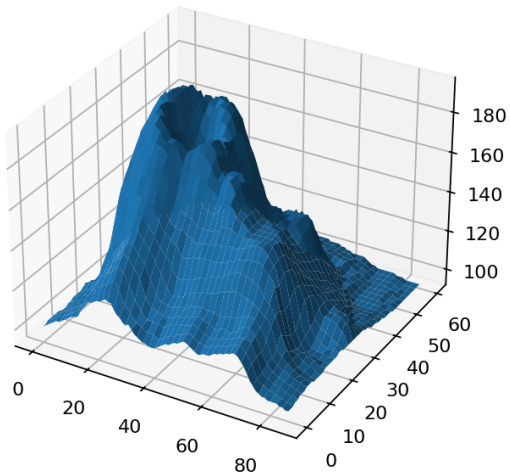
Python Code: 3D Surface Plot of the Volcano Data

```
ny, nx = volcano.shape
X, Y = np.meshgrid(np.arange(nx), np.arange(ny))

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, volcano)

plt.savefig("../figs/3-10.png", dpi=160, bbox_inches='tight')
plt.show()
```

Figure: 3D Surface Plot of Volcano Data



Filled Contour Plots

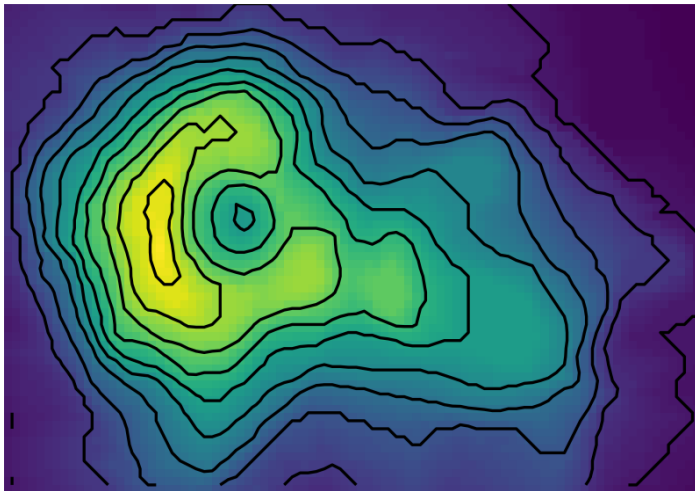
- A contour plot with a 3D effect can be displayed in 2D by overlaying contour lines on a color map of the height.
- Using an image-style color background without contour lines yields a display similar in spirit to filled contour plots.

```
plt.figure()
plt.imshow(volcano, origin="lower", aspect="equal")
plt.axis("off")

levels = np.arange(100, 201, 10)
plt.contour(volcano, levels=levels, colors="black", origin="lower")

plt.show()
```


Figure: Image Display with Overlaid Contours



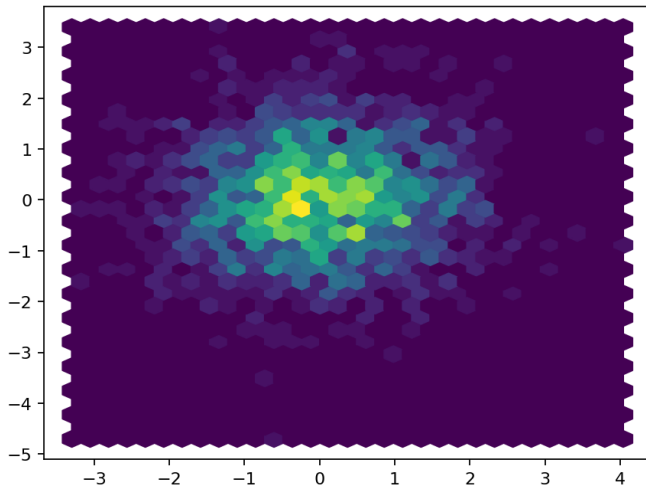
2D Histogram (Flat Density Histogram)

- In large data sets, 2D scatterplots can hide structure when some regions are very dense.
- A 2D (flat) histogram represents the density in each bin using color intensity.
- Example: simulated standard bivariate normal data displayed with hexagonal bins.

```
x = np.random.randn(2000, 2)

plt.figure()
plt.hexbin(x[:, 0], x[:, 1], gridsize=30)
plt.show()
```

Figure: Flat Density Histogram with Hexagonal Bins



Square-bin Variant (2D Histogram)

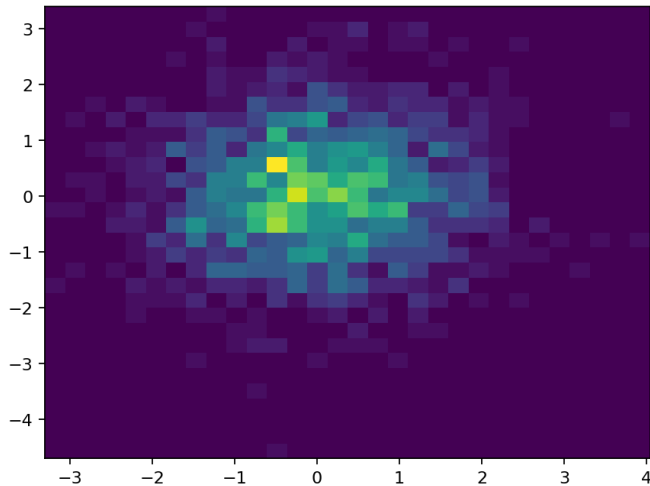
- A similar display can use square bins and a color palette to represent density per bin.

```
import numpy as np
import matplotlib.pyplot as plt

x = np.random.randn(2000, 2)

plt.figure()
plt.hist2d(x[:, 0], x[:, 1], bins=30)
plt.show()
```

Figure: 2D Histogram with Square Bins



Other 2D Representations of Data

- Beyond contour plots and other 2D projections, there are other methods for representing multivariate data in two dimensions.
- These include Andrews curves, parallel coordinate plots, and iconographic displays (e.g., segment plots, star plots).

- For $X_1, \dots, X_n \in \mathbb{R}^d$, Andrews curves map each observation $x_i = (x_{i1}, \dots, x_{id})$ to a function $f_i(t)$.
- The mapping uses an orthogonal basis $\{2^{-1/2}, \sin(kt), \cos(kt)\}$.
- Differences are amplified more in lower-frequency terms, so the representation depends on the order of variables/features.

$$f_i(t) = \frac{x_{i1}}{\sqrt{2}} + x_{i2} \sin t + x_{i3} \cos t + x_{i4} \sin 2t + x_{i5} \cos 2t + \dots, \quad -\pi \leq t \leq \pi.$$

Python Code: Andrews Curves (Leaf Measurements Example)

```
leaf = pd.read_csv("data/leafshape17.csv", index_col=0)
X = leaf[["bladelen", "petiole", "bladewid"]].to_numpy()

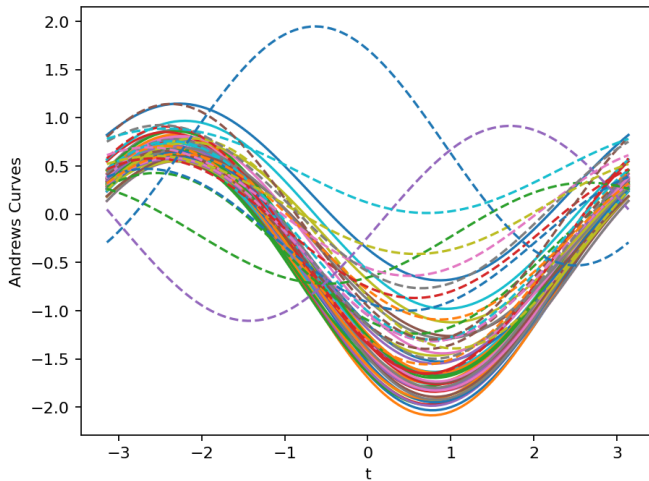
# scale each column to range [-1, 1]
mins = X.min(axis=0)
maxs = X.max(axis=0)
X_scaled = 2 * (X - mins) / (maxs - mins) - 1

t = np.linspace(-np.pi, np.pi, 101)

plt.figure()
for i in range(X_scaled.shape[0]):
    v = X_scaled[i, :]
    y = v[0]/np.sqrt(2) + v[1]*np.sin(t) + v[2]*np.cos(t)
    lty = "-" if leaf["arch"].iloc[i] == 0 else "--"
    plt.plot(t, y, lty)

plt.xlabel("t")
plt.ylabel("Andrews Curves")
plt.show()
```

Figure: Andrews Curves by Leaf Architecture



- The plot reveals similarities within the two leaf architecture groups and differences between these groups.
- In general, this type of plot may reveal possible clustering of data.

Parallel Coordinate Plots

- Parallel coordinate plots visualize vectors in $\mathbb{R}^d$ using d equidistant parallel axes.
- Each coordinate is plotted on its corresponding axis, and the points are joined by line segments.
- Axes are often shown as parallel lines (commonly horizontal in this representation).
- Example: a panel display for the crabs data (groups by species and sex), using black and white and a subset for readability.

Python Code: Parallel Coordinate Panel Display (Subset of Crabs)

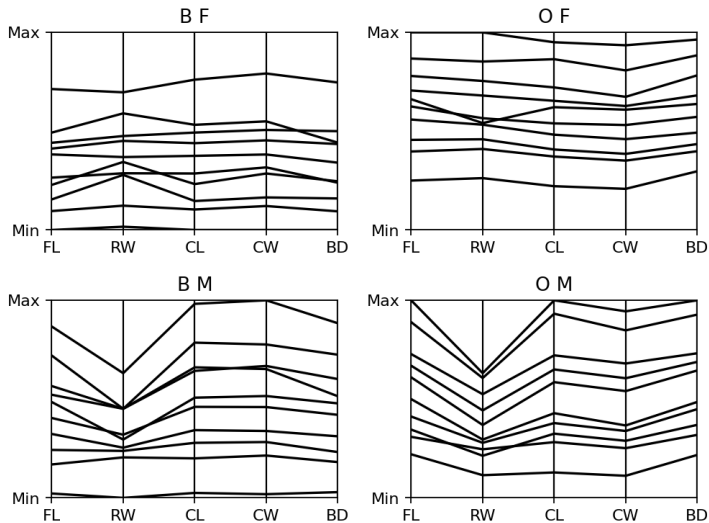
```
from pandas.plotting import parallel_coordinates

crabs = pd.read_csv("data/crabs.csv", index_col=0); x = crabs.iloc[4::5].copy() # 5, 10, ...
vars = ["FL", "RW", "CL", "CW", "BD"]
mins = x[vars].min(axis=0); maxs = x[vars].max(axis=0)
x[vars] = (x[vars] - mins) / (maxs - mins)

fig, axes = plt.subplots(2, 2)
sp_levels = ["B", "O"]; sex_levels = ["F", "M"]
for i, sp in enumerate(sp_levels):
    for j, sex in enumerate(sex_levels):
        ax = axes[j, i]
        sub = x[(x["sp"] == sp) & (x["sex"] == sex)][vars].copy(); sub["grp"] = "all" #dummy
        parallel_coordinates(sub, "grp", ax=ax, color=["k"])
        ax.get_legend().remove(); ax.set_title(f"{sp} {sex}")
        ax.set_ylim(0, 1); ax.set_yticks([0, 1]); ax.set_yticklabels(["Min", "Max"])

plt.tight_layout(); plt.show()
```

Figure: Parallel Coordinate Plots by Species and Sex



- The axes correspond to the five measurements (frontal lobe size, rear width, carapace length, carapace width, body depth).
- Much of the variability between groups is in overall size.

Segments, Stars, and Other Representations

- Multivariate data can be represented by a two-dimensional icon (glyph), such as a star.
- These icon displays are best shown in a table so observations can be compared.
- They are not very practical for high dimension or large data sets, but can be useful for small data.
- Example: a radar (star) plot for a subset of the crabs data (males), using size-adjusted measurements.

Python Code: Radar (Star) Plot for Size-adjusted Male Crabs

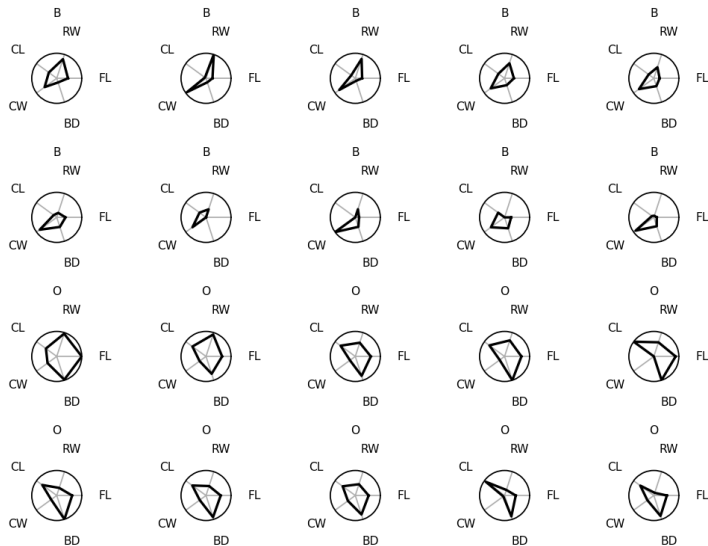
```
x = crabs.iloc[4::5].query("sex=='M'")
vars = ["FL", "RW", "CL", "CW", "BD"]
a = x["CW"] * x["CL"]
xs = x[vars].div(np.sqrt(a), axis=0); xs = (xs - xs.min()) / (xs.max() - xs.min())

n, nrow = len(xs), 4; ncol = int(np.ceil(n / nrow))
ang = np.linspace(0, 2*np.pi, len(vars), endpoint=False); ang = np.r_[ang, ang[0]]
fig, axes = plt.subplots(nrow, ncol, subplot_kw=dict(projection="polar"))
axes = np.array(axes).reshape(nrow, ncol)

for ax, k in zip(axes.ravel(), range(nrow*ncol)):
    ax.set(ylim=(0, 1)); ax.set_yticks([])
    ax.set_xticks(ang[:-1]); ax.set_xticklabels(vars, fontsize=7)
    v = xs.iloc[k].to_numpy()
    ax.plot(ang, np.r_[v, v[0]], color="k")
    ax.set_title(str(x["sp"].iloc[k]), fontsize=7)

plt.tight_layout(); plt.show()
```

Figure: Segment (Star) Plot for Male Crabs



- Differences between the species (for males) in this sample are evident in the segment plot.
- The plot suggests, for example, that orange crabs have greater body depth relative to carapace width than blue crabs.

Principal Components Analysis

Principal Components Analysis

- PCA is a *dimension reduction* method for multivariate data with many (often correlated) variables.
- It provides an approximate representation of the data in a lower-dimensional space.
- PCA for visualization uses *projections*: project data onto directions that explain the most variation.
- The first principal component (PC1) is the direction of maximal variance; subsequent PCs maximize variance subject to orthogonality.
- If most variation is captured by the first $k < p$ PCs, then $(Z_1, \dots, Z_k) \in \mathbb{R}^k$ approximates $X \in \mathbb{R}^p$.

PCA via Variance-maximizing Projections

- Let $X = (X_1, \dots, X_p)^\top$, $E[X] = \mu$, $\text{Cov}(X) = \Sigma$.
- Consider $Y = w^\top (X - \mu)$ with $\|w\| = 1$; choose w to maximize $\text{Var}(Y)$.
- Spectral decomposition: $\Sigma = P\Lambda P^\top$, with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$.
- PC1: $Z_1 = v_1^\top (X - \mu)$, where v_1 is the eigenvector of λ_1 ; then $\text{Var}(Z_1) = \lambda_1$.
- PC*i*: $Z_i = v_i^\top (X - \mu)$, $i = 1, \dots, p$, and the Z_i are uncorrelated.

- $\text{Var}(Z_i) = \lambda_i$ and $\text{Var}(Z_1) \geq \dots \geq \text{Var}(Z_p) \geq 0$.
- $\text{Cor}(Z_i, Z_j) = 0$ for $i \neq j$.
- Fraction of variance captured by first k PCs:

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^p \lambda_j}.$$

- The coordinates of eigenvectors v_i are called *loadings*.
- PCA is not scale-invariant: if variables are on different scales, scale to common variance before computing PCs.

Sample PCA and Covariance Estimators

- In practice, replace μ by the sample mean and Σ by a sample covariance matrix.
- Two common estimators: one divides by n (ML estimator), another divides by $n - 1$ (unbiased estimator).
- Different software may use different estimators, which can lead to small differences in results.

Example: Exam Scores (Open/Closed Book)

- Data: five exam scores for 88 students (mechanics, vectors, algebra, analysis, statistics).
- Mechanics and vectors are closed book; the other three are open book.
- Pairwise scatterplots/correlations show several highly correlated variables.
- Goal: use PCA to summarize relationships with uncorrelated components and assess whether 2–3 PCs suffice for visualization.

Python Code: Correlations and Eigen-decomposition (Standardized Data)

```
scor = pd.read_csv("data/scor.csv", index_col=0)

X = (scor - scor.mean()) / scor.std(ddof=0)

# covariance of standardized data and eigen-decomposition
S = np.cov(X.to_numpy(), rowvar=False, ddof=0)
lam, P = np.linalg.eigh(S) # ascending order
lam = lam[::-1] # descending eigenvalues
P = P[:, ::-1] # corresponding eigenvectors (loadings)

# pairwise correlations
corr = X.corr()
print(corr.round(3))
```

Python Code: Scree Plot + Proportion/Cumulative Variance Table

```
prop = lam / lam.sum()
cum = np.cumsum(prop)

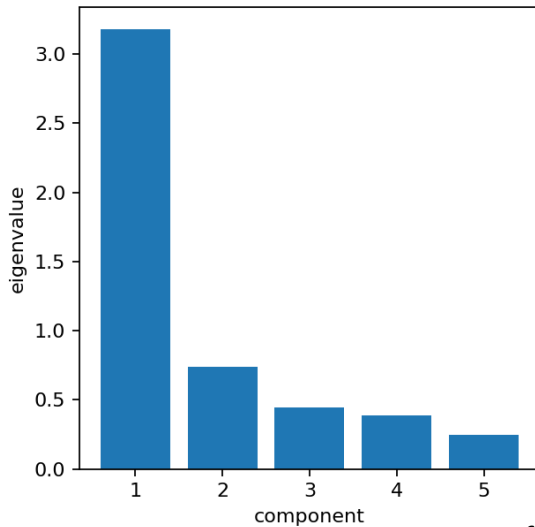
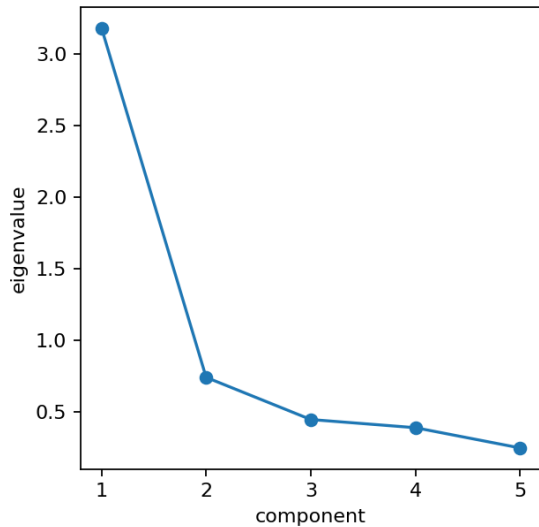
# scree plot (eigenvalues)
fig, axes = plt.subplots(1, 2, figsize=(8,4))
axes[0].plot(np.arange(1, len(lam) + 1), lam, marker="o")
axes[0].set_xlabel("component")
axes[0].set_ylabel("eigenvalue")

# bar plot variant
axes[1].bar(np.arange(1, len(lam) + 1), lam)
axes[1].set_xlabel("component")
axes[1].set_ylabel("eigenvalue")

plt.tight_layout()
plt.show()

tab = np.vstack([prop, cum])
print(np.round(tab, 6))
```

Figure: Scree Plot of Eigenvalues



From Eigenvectors to Principal Components

- Let X be the centered and scaled data matrix and P the matrix of standardized eigenvectors.
- The principal component score matrix is $Z = XP$.
- The columns of Z are the PCs (PC1, PC2, ...).

```
Z = X.to_numpy() @ P # scores (n x p)
```

```
print(Z.shape)
```

```
print(np.round(Z[:6, :], 6)) # first rows of PC score matrix
```

Software PCA Output and Sign Ambiguity

- PCA software typically returns loadings and scores; loadings correspond to eigenvectors.
- Eigenvectors (and PCs) are unique only up to sign, so results may match except for sign flips.

```
from sklearn.decomposition import PCA

pca = PCA()
scores = pca.fit_transform(X.to_numpy()) # PC scores
loadings = pca.components_.T # columns ~ eigenvectors (up to sign)

print("explained variance ratio:", np.round(pca.explained_variance_ratio_, 6))
print("cumulative:", np.round(np.cumsum(pca.explained_variance_ratio_), 6))
```

PC Biplot (First Two PCs)

- A PC biplot displays the data in the $(PC1, PC2)$ coordinate system and adds loading vectors.
- This gives a low-dimensional view for visualization and interpretation of components.
- (Python note) A built-in one-call `biplot` is uncommon; below is a minimal biplot-style implementation.

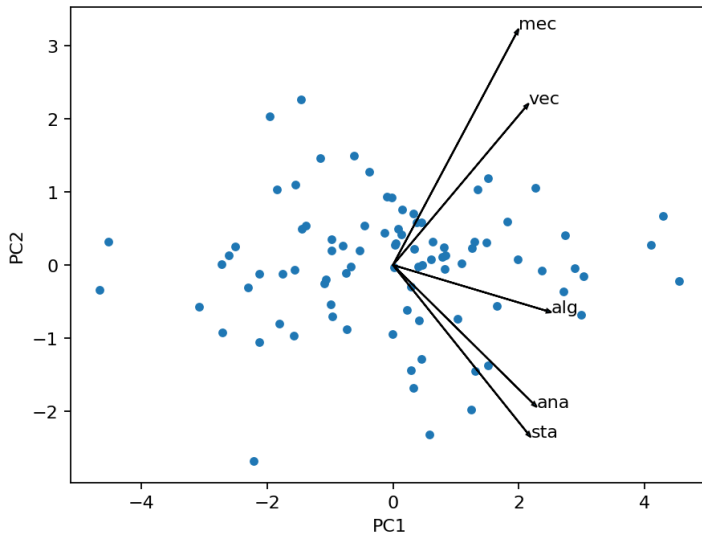
Python Code: Minimal PC Biplot (Scores + Loading Vectors)

```
plt.figure()
plt.scatter(scores[:, 0], scores[:, 1], s=15)
cols = list(scor.columns)

scale = 5.0
for j in range(loadings.shape[0]):
    plt.arrow(0, 0, scale*loadings[j, 0], scale*loadings[j, 1], length_includes_head=True,
              head_width=0.05)
    plt.text(scale*loadings[j, 0], scale*loadings[j, 1], cols[j])

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```


Figure: PC Biplot of Exam Scores



Interpretation from the Biplot and Loadings

- The algebra loading has almost zero slope with respect to PC2, indicating algebra is lightly weighted in PC2.
- Statistics and analysis have positive weight on PC2, while mechanics and vectors have negative weight in the PC2 direction.
- Algebra has its highest loading on PC5 (a component explaining less than 5% of total variance).